

BGauss B2B Buyback — Complete Deployment Guide

EC2 (Windows) + IIS + RDS SQL Server + Flutter APK

Using Elastic IP only — No Domain Required

ARCHITECTURE

```
Flutter APK → http://54.x.x.x:80 → EC2 Windows Server
                                   IIS → ASP.NET Core API
                                   ↓
                                   RDS SQL Server
                                   (epc-db.ccbwwugws73u.us-east-
1.rds.amazonaws.com)
```

Your Elastic IP (assign once, never changes):

After completing Part 1, note your Elastic IP — use it everywhere in this guide as

`YOUR_ELASTIC_IP`.

PART 1 — AWS EC2 INSTANCE SETUP

1.1 Launch EC2 Instance

1. AWS Console → **EC2** → **Launch Instance**
2. Settings:
 - Name: `bgauss-api-server`
 - AMI: **Windows Server 2022 Base** (64-bit x86)
 - Instance type: **t3.medium** (minimum — 2 vCPU, 4GB RAM)
 - Key pair: Create new → `bgauss-key` → **Download .pem** — save securely
 - Storage: **30 GB gp3**
3. Click **Launch Instance**

1.2 Security Group Inbound Rules

In the security group created with the instance, set these rules:

Type	Port	Source	Purpose
RDP	3389	Your IP only	Remote Desktop to EC2
HTTP	80	0.0.0.0/0	Flutter APK → API
Custom TCP	1433	This SG only	EC2 → RDS (SQL Server)

Port 80 is enough — no HTTPS/domain needed for internal dealer app.

1.3 Assign Elastic IP (Fixed — Never Changes)

1. EC2 → **Elastic IPs** → **Allocate Elastic IP address** → Allocate
2. Select the new IP → **Actions** → **Associate Elastic IP address**
3. Instance: select `bgauss-api-server` → Associate
4. **Note this IP** — this is your permanent API address, e.g. `54.123.45.67`

1.4 Allow EC2 to Reach RDS

1. Go to **RDS** → **your DB instance** → scroll to **VPC security groups** → click it
2. **Inbound rules** → **Edit** → **Add rule:**
 - Type: `MS SQL`
 - Port: `1433`
 - Source: **Security group ID of your EC2** (not IP — search for `bgauss-api-server` SG)
3. Save

PART 2 — CONNECT TO EC2 VIA REMOTE DESKTOP

2.1 Get Windows Password

```
EC2 → Instances → select your instance
→ Actions → Security → Get Windows password
→ Upload bgauss-key.pem → Decrypt Password
→ Copy the password
```

2.2 Open Remote Desktop (RDP)

On your Windows PC:

- Press `Win + R` → type `mstsc` → Enter

- Computer: `YOUR_ELASTIC_IP` (e.g. `54.123.45.67`)
- Username: `Administrator`
- Password: (from above)
- Click Connect → accept certificate

You are now inside the EC2 server.

PART 3 — INSTALL SOFTWARE ON EC2

Open **PowerShell as Administrator** inside the RDP session (right-click Start → Windows PowerShell (Admin)).

3.1 Install IIS

powershell

```
Install-WindowsFeature -Name Web-Server, Web-Asp-Net45, Web-Net-Ext45, `
    Web-ISAPI-Ext, Web-ISAPI-Filter, Web-Mgmt-Console, Web-Scripting-Tools, `
    NET-Framework-45-Core, NET-Framework-45-ASPNET -IncludeManagementTools

Write-Host "IIS installed" -ForegroundColor Green
```

3.2 Install .NET 8 ASP.NET Core Hosting Bundle

Manual steps inside RDP:

1. Open **Internet Explorer** or **Edge** on EC2
2. Go to: `https://dotnet.microsoft.com/en-us/download/dotnet/8.0`
3. Under **ASP.NET Core Runtime 8.x** → click **Hosting Bundle** → Windows
4. Download and **run the .exe installer** → Next → Install → Finish

Verify in PowerShell:

powershell

```
dotnet --version
# Should show: 8.x.x
```

3.3 Restart IIS to Load .NET Module

powershell

```
iisreset /restart  
Write-Host "IIS restarted" -ForegroundColor Green
```

3.4 Install SSMS (to verify DB connection)

1. In Edge on EC2, go to: <https://aka.ms/ssmsfullsetup>
2. Download and install SSMS → Next → Install (takes ~5 min)

PART 4 — VERIFY RDS CONNECTION FROM EC2

Open **SQL Server Management Studio (SSMS)**:

```
Server name:      epc-db.ccbwwugws73u.us-east-1.rds.amazonaws.com,1433  
Authentication:  SQL Server Authentication  
Login:           admin  
Password:        Admin-123-456
```

Click **Connect**.

✓ If connected — RDS is reachable from EC2.

✗ If fails — recheck Part 1.4 (RDS security group must allow EC2's SG on 1433).

PART 5 — BUILD API ON YOUR LOCAL MACHINE

On your **local development PC** (not inside RDP):

5.1 Update appsettings.json

Your `backend\B2BBuyback.Api\appsettings.json` already has the correct RDS connection string. Confirm it looks like:

```
json
```

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=epc-db.ccbwwugws73u.us-east-1.rds.amazonaws.com,143
  },
  "JwtSettings": {
    "Key": "ThisIsMySuperSecretKeyForJwt12345",
    "Issuer": "BGAUSS.Api",
    "Audience": "BGAUSS.Client"
  },
  "Smtp": {
    "Host": "smtp.office365.com",
    "Port": "587",
    "User": "priyanka.nikam@bgauss.com",
    "Password": "PN@bg052026",
    "From": "BGauss Exchange <priyanka.nikam@bgauss.com>"
  },
  "Exchange": {
    "AdminEmail": "priyanka.nikam@bgauss.com"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Warning",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

5.2 Confirm Program.cs CORS (for APK — no domain)

Your `Program.cs` must use `AllowAnyOrigin()` — **not** `WithOrigins(...)`:

```

csharp

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", policy =>
        policy.AllowAnyOrigin() // APK has no domain
            .AllowAnyHeader()
            .AllowAnyMethod());
    // DO NOT add .AllowCredentials() with AllowAnyOrigin
});

```

5.3 Publish Release Build

In PowerShell on your **local machine**:

```
powershell

cd C:\Users\priyanka.nikam\Desktop\B2BBUYBACKANDROID\b2bbuybackandriod\backend\B2BBu

dotnet publish -c Release -r win-x64 --self-contained false `
  -o "C:\bgauss-publish"

Write-Host "Publish complete - files at C:\bgauss-publish" -ForegroundColor Green
```

This creates a folder `C:\bgauss-publish` with all API files ready to copy to EC2.

5.4 Apply EF Migrations to RDS (if schema changed)

```
powershell

cd C:\Users\priyanka.nikam\Desktop\B2BBUYBACKANDROID\b2bbuybackandriod\backend\B2BBu

dotnet ef database update

# If dotnet-ef not installed:
dotnet tool install --global dotnet-ef
dotnet ef database update
```

PART 6 — COPY FILES TO EC2

Option A — Drag and Drop via RDP (Easiest)

1. In your RDP window, make the window smaller so you can see both your PC and the RDP
2. On EC2, open File Explorer → go to `C:\inetpub\`
3. Create folder: `C:\inetpub\bgauss-api`
4. On your **local PC**, open `C:\bgauss-publish`
5. Select all files → drag into the RDP window → drop into `C:\inetpub\bgauss-api`

Wait for all files to copy.

Option B — Via PowerShell (Faster for large files)

On your **local machine**, compress first:

```
powershell
```

```
Compress-Archive -Path "C:\bgauss-publish\*" `
  -DestinationPath "C:\bgauss-publish.zip"
```

Then in your RDP session on EC2:

```
powershell
```

```
# Create destination
```

```
New-Item -ItemType Directory -Force -Path "C:\inetpub\bgauss-api"
```

```
# If you copied the zip via RDP clipboard:
```

```
Expand-Archive -Path "C:\Users\Administrator\Desktop\bgauss-publish.zip" `
  -DestinationPath "C:\inetpub\bgauss-api" -Force
```

```
Write-Host "Files extracted" -ForegroundColor Green
```

6.1 Create Required Folders

Run this inside the **RDP PowerShell**:

```
powershell
```

```
# Create folders the API needs
```

```
New-Item -ItemType Directory -Force -Path "C:\inetpub\bgauss-api\wwwroot"
```

```
New-Item -ItemType Directory -Force -Path "C:\inetpub\bgauss-api\wwwroot\ExchangeIma
```

```
New-Item -ItemType Directory -Force -Path "C:\inetpub\bgauss-api\wwwroot\CaseDocumen
```

```
New-Item -ItemType Directory -Force -Path "C:\inetpub\bgauss-api\wwwroot\ScootyInven
```

```
New-Item -ItemType Directory -Force -Path "C:\inetpub\bgauss-api\logs"
```

```
Write-Host "Folders created" -ForegroundColor Green
```

6.2 Set IIS Permissions

```
powershell
```

```
# Read permissions for API files
icacls "C:\inetpub\bgauss-api" /grant "IIS_IUSRS:(OI)(CI)RX" /T

# Full write permissions for uploads folder
icacls "C:\inetpub\bgauss-api\wwwroot" /grant "IIS_IUSRS:(OI)(CI)F" /T
icacls "C:\inetpub\bgauss-api\wwwroot" /grant "NETWORK SERVICE:(OI)(CI)F" /T

# Write permissions for logs
icacls "C:\inetpub\bgauss-api\logs" /grant "IIS_IUSRS:(OI)(CI)F" /T

Write-Host "Permissions set" -ForegroundColor Green
```

PART 7 — CREATE web.config

In the RDP session, create this file at `C:\inetpub\bgauss-api\web.config`:

```
powershell
```



```
$webconfig = @'
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <location path="." inheritInChildApplications="false">
    <system.webServer>
      <handlers>
        <add name="aspNetCore" path="*" verb="*"
          modules="AspNetCoreModuleV2"
          resourceType="Unspecified" />
      </handlers>
      <aspNetCore processPath="dotnet"
        arguments=".\B2BBuyback.Api.dll"
        stdoutLogEnabled="true"
        stdoutLogFile=".\logs\stdout"
        hostingModel="inprocess">
        <environmentVariables>
          <environmentVariable name="ASPNETCORE_ENVIRONMENT" value="Production" />
        </environmentVariables>
      </aspNetCore>
    </system.webServer>
  </location>
</configuration>
'@

Set-Content -Path "C:\inetpub\bgauss-api\web.config" -Value $webconfig
Write-Host "web.config created" -ForegroundColor Green
```

PART 8 — CONFIGURE IIS

8.1 Create Application Pool

Open **IIS Manager** (Start → search "IIS Manager" → open):

1. Left panel → **Application Pools** → **Add Application Pool** (right-click)
 - Name: **BGaussApiPool**
 - .NET CLR version: **No Managed Code**
 - Managed pipeline mode: **Integrated**
 - Click OK
2. Right-click **BGaussApiPool** → **Advanced Settings**:
 - Identity → change to **LocalSystem**

- Start Mode: **AlwaysRunning**
- Idle Time-out (minutes): **0**
- Regular Time Interval (minutes): **0**
- Click OK

8.2 Create Website

1. Left panel → **Sites** → **right-click** → **Add Website**:

- Site name: `BGaussApi`
- Application pool: click Select → choose `BGaussApiPool`
- Physical path: `C:\inetpub\bgauss-api`
- Binding: HTTP, All Unassigned, Port **80**
- Click OK

2. If **Default Web Site** is using port 80, stop it first:

- Right-click **Default Web Site** → **Stop**

8.3 Test API on EC2 Locally

In the RDP session, open **Edge** and go to:

```
http://localhost/swagger
```

✓ You should see the Swagger UI page.

If you see a **502 Bad Gateway**:

- Check that .NET 8 Hosting Bundle is installed
- Run `iisreset /restart` and try again
- Check logs: `Get-Content "C:\inetpub\bgauss-api\logs\stdout*.log" -Tail 30`

If you see **403 Forbidden**:

- Re-run the `icacls` commands from Part 6.2

PART 9 — TEST SWAGGER FROM OUTSIDE EC2

On **your local PC browser** (not inside RDP), open:

```
http://YOUR_ELASTIC_IP/swagger
```

✓ Swagger UI loads — API is publicly accessible.

If it doesn't load:

- Check AWS Security Group has port 80 open to `0.0.0.0/0`
- Check Windows Firewall on EC2 (run in RDP PowerShell):

```
powershell

# Allow HTTP port 80 through Windows Firewall
netsh advfirewall firewall add rule name="HTTP 80" `
  dir=in action=allow protocol=TCP localport=80

# Allow HTTPS port 443 (future use)
netsh advfirewall firewall add rule name="HTTPS 443" `
  dir=in action=allow protocol=TCP localport=443

iisreset /restart
```

PART 10 — TEST API WITH POSTMAN OR BROWSER

Test these URLs from your browser (replace with your Elastic IP):

<code>http://YOUR_ELASTIC_IP/swagger</code>	→ Swagger UI
<code>http://YOUR_ELASTIC_IP/api/ScootyInventory/models-list</code>	→ returns JSON
<code>http://YOUR_ELASTIC_IP/api/DealerAuth/send-otp</code>	→ 405 (POST only – correct)

PART 11 — UPDATE FLUTTER appsettings AND BUILD RELEASE APK

11.1 Update `api_service.dart` default value

In `frontend/lib/services/api_service.dart`, update the default:

```
dart

static const String baseUrl = String.fromEnvironment(
  'API_BASE_URL',
  defaultValue: 'http://YOUR_ELASTIC_IP', // ← update this
);
```

11.2 Add cleartext traffic permission (HTTP over Elastic IP)

Since you're using HTTP (not HTTPS), add to

`frontend/android/app/src/main/AndroidManifest.xml`

inside the `<application>` tag:

```
xml

<application
  android:usesCleartextTraffic="true"
  android:label="BGauss PI"
  ...>
```

11.3 Update app version in pubspec.yaml

```
yaml

version: 1.0.1+2  # increment build number each release
```

11.4 Generate Signing Keystore (One-Time Setup)

Run this **once** on your local machine — save the .jks file permanently:

```
powershell

cd C:\Users\priyanka.nikam\Desktop\B2BBUYBACKANDROID\b2bbuybackandriod\frontend

keytool -genkey -v -keystore bgauss-release.jks `
  -keyalg RSA -keysize 2048 -validity 10000 `
  -alias bgauss-key `
  -dname "CN=BGauss, OU=IT, O=BGauss Auto Pvt Ltd, L=Mumbai, ST=Maharashtra, C=IN"

# When prompted:
# Enter keystore password: bgauss@2026
# Re-enter: bgauss@2026
# Key password (press Enter to use same): bgauss@2026
```

11.5 Create key.properties

Create file `frontend/android/key.properties` (do NOT commit to git):

```
properties
```

```
storePassword=bgauss@2026
keyPassword=bgauss@2026
keyAlias=bgauss-key
storeFile=../../bgauss-release.jks
```

11.6 Update android/app/build.gradle

Open `frontend/android/app/build.gradle` and add the signing config.

At the top of the file (after the first line), add:

```
groovy

def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}
```

Inside `android { ... }`, add signingConfigs and update buildTypes:

```
groovy

android {
    // ... existing compileSdk, defaultConfig etc ...

    signingConfigs {
        release {
            keyAlias keystoreProperties['keyAlias']
            keyPassword keystoreProperties['keyPassword']
            storeFile keystoreProperties['storeFile'] ? file(keystoreProperties['storeFile']) : null
            storePassword keystoreProperties['storePassword']
        }
    }

    buildTypes {
        release {
            signingConfig signingConfigs.release
            minifyEnabled false
            shrinkResources false
        }
    }
}
```

11.7 Build Release APK

```
powershell

cd C:\Users\priyanka.nikam\Desktop\B2BBUYBACKANDROID\b2bbuybackandriod\frontend

flutter build apk --release `
  --dart-define="API_BASE_URL=http://YOUR_ELASTIC_IP"

# Example with real IP:
# flutter build apk --release --dart-define="API_BASE_URL=http://54.123.45.67"
```

11.8 Find the APK

```
frontend\build\app\outputs\flutter-apk\app-release.apk
```

This is the final APK to distribute to dealer phones.

PART 12 — INSTALL APK ON DEALER PHONES

Method A — Direct USB Install

```
powershell

# List connected devices
flutter devices

# Install directly
flutter install --release
# OR
adb install build\app\outputs\flutter-apk\app-release.apk
```

Method B — WhatsApp / Email Share

1. Send `app-release.apk` via WhatsApp or email to the dealer
2. On dealer phone → open the file → **Install**
3. If blocked: Settings → Security → **Allow installation from unknown sources**

Method C — Google Drive Link

1. Upload APK to Google Drive

2. Share link → dealer downloads and installs

PART 13 — RE-DEPLOY API AFTER CODE CHANGES

Every time you update the backend code:

Step 1 — On local machine:

```
powershell

cd backend\B2BBuyback.Api

# Rebuild
dotnet publish -c Release -r win-x64 --self-contained false `
  -o "C:\bgauss-publish"
```

Step 2 — On EC2 (via RDP PowerShell):

```
powershell

# Stop the site
Stop-WebSite -Name "BGaussApi"

# Copy new files (after dragging zip or using RDP clipboard)
# Expand-Archive -Path "C:\bgauss-publish-new.zip" -DestinationPath "C:\inetpub\bgauss"

# Start site
Start-WebSite -Name "BGaussApi"

Write-Host "Deployment complete" -ForegroundColor Green
```

Step 3 — Verify:

`http://YOUR_ELASTIC_IP/swagger`

PART 14 — RE-BUILD APK AFTER FLUTTER CODE CHANGES

Every time you update the Flutter code:

powershell

```
cd frontend
```

```
flutter build apk --release `
  --dart-define="API_BASE_URL=http://YOUR_ELASTIC_IP"
```

Distribute the new APK from `build\app\outputs\flutter-apk\app-release.apk`.

PART 15 — TROUBLESHOOTING

Problem	Cause	Fix
Swagger shows 502 Bad Gateway	.NET 8 Hosting Bundle missing or wrong version	Reinstall hosting bundle, run <code>iisreset</code>
Swagger shows 403 Forbidden	IIS permissions wrong	Re-run <code>icacls</code> commands from Part 6.2
Swagger shows 500 Error	App crash / DB connection fail	Check <code>C:\inetpub\bgauss-api\logs\stdout*.log</code>
Can't reach RDS from EC2	RDS SG doesn't allow EC2 SG	Add inbound rule in RDS SG: port 1433, source = EC2 SG
APK shows "Cannot reach server"	Wrong Elastic IP in APK	Rebuild APK with correct <code>--dart-define</code>
APK timeout on HTTP	<code>usesCleartextTraffic</code> missing	Add <code>android:usesCleartextTraffic="true"</code> to AndroidManifest
Image upload fails	wwwroot folder permissions	<code>icacls "C:\inetpub\bgauss-api\wwwroot" /grant "IIS_IUSRS:(OI)(CI)F" /T</code>
IIS site won't start	Port 80 conflict with Default Web Site	Stop Default Web Site first
APK installs but crashes	API URL unreachable	Test <code>http://YOUR_ELASTIC_IP/swagger</code> from phone browser first

Check Logs on EC2

powershell

Application stdout logs

`Get-Content "C:\inetpub\bgauss-api\logs\stdout*.log" -Tail 50`

Windows Event Log

`Get-EventLog -LogName Application -Newest 20 | Where-Object {$_.Source -like "*IIS*"}`

FINAL CHECKLIST

AWS

- ✓ EC2 Windows Server 2022 launched
- ✓ Elastic IP assigned and associated
- ✓ Security group: port 80 open to 0.0.0.0/0, port 3389 to your IP only
- ✓ RDS security group allows EC2 SG on port 1433

EC2 SOFTWARE

- ✓ IIS installed (Install-WindowsFeature)
- ✓ .NET 8 ASP.NET Core Hosting Bundle installed
- ✓ iisreset /restart done after installing hosting bundle
- ✓ SSMS installed and RDS connection verified

API DEPLOYMENT

- ✓ dotnet publish -c Release -r win-x64 run locally
- ✓ Files copied to C:\inetpub\bgauss-api
- ✓ wwwroot, ExchangeImages, CaseDocuments, logs folders created
- ✓ icacls permissions set for IIS_IUSRS
- ✓ web.config created with AspNetCoreModuleV2 config
- ✓ IIS App Pool: BGaussApiPool (No Managed Code, LocalSystem)
- ✓ IIS Site: BGaussApi on port 80, physical path C:\inetpub\bgauss-api
- ✓ Default Web Site stopped (if conflicting on port 80)
- ✓ Windows Firewall port 80 allowed
- ✓ http://localhost/swagger works inside EC2 RDP
- ✓ http://YOUR_ELASTIC_IP/swagger works from outside

FLUTTER APK

- ✓ AndroidManifest.xml has android:usesCleartextTraffic="true"
- ✓ bgauss-release.jks keystore created and stored safely

- ✓ android/key.properties created
- ✓ android/app/build.gradle signingConfigs updated
- ✓ flutter build apk --release --dart-define="API_BASE_URL=http://YOUR_ELASTIC_IP"
- ✓ app-release.apk installed on dealer phones
- ✓ Login works end-to-end on released APK

BGauss B2B Buyback Deployment — May 2026